

全栈工程师 AI 编程面试题

60 分钟

说明：本面试题面向全栈工程师岗位，适配 React 前端+Java 后端+PostgreSQL 数据库技术栈，结合企业核心业务场景（客户订单管理）。要求 60 分钟内完成（编程实现 45 分钟，讲解+答疑 15 分钟），面试者需通过 AI 编程（可使用 GPT、Claude、Genimin、Qwen 等 AI 工具辅助，但需明确标注 AI 辅助场景及自身修改逻辑）完成。重点考察技术栈熟练度、业务场景落地能力、高效问题解决能力、代码规范及全栈思维，所有代码需可运行，附带关键注释，同时简要回答题目后的拓展问题，面试时需讲解代码逻辑、业务适配思路及 AI 辅助过程。

面试时长：60 分钟（编程实现 45 分钟，讲解+答疑 15 分钟）

技术栈要求：前端（React18+JavaScript/TypeScript+Redux Toolkit+React Router+Ant Design）、后端（Java+Spring Boot+Spring Security+MyBatis-Plus）、数据库（PostgreSQL+Redis）、接口联调（Axios）

一、前端模块（25 分）

需求：实现“客户订单管理页面”，贴合真实业务场景，要求 UI 规范、交互流畅、兼容 Chrome 最新版本，符合前端工程化规范，使用 AI 辅助优化代码结构、性能及交互体验，需在 20 分钟内完成核心实现。

1. 页面布局：左侧业务侧边栏（包含系统首页、客户管理、订单管理、销售跟进、设备关联 5 个菜单，支持折叠/展开，添加过渡动画）、顶部导航栏（显示登录账号、业务模块切换、退出登录按钮，添加下拉菜单）、主体内容区（自适应屏幕尺寸，支持响应式布局，区分业务模块样式）。

1. 核心业务功能（重点考察业务落地，非通用功能）：聚焦客户订单全流程管理，结合真实业务场景设计，涵盖订单展示、新增编辑、审核删除及详情查看，突出数据联动和业务规则校验，避免通用化功能设计，考察业务落地能力。

2. 订单列表展示（核心）：展示系统中客户订单数据，包含订单 ID、客户名称（关联客户信息）、订单类型（采购/销售）、订单金额、交货日期、设备编号（关联设备信息）、订单状态（待审核/已审核/已发货/已完成），支持分页（每页 8 条）、多条件筛选（按订单类型/状态/交货日期区间）、模糊搜索（按订单 ID/客户名称/设备编号）。

3. 订单新增/编辑（合并弹窗，复用表单组件）：弹窗表单，核心字段包含：客户选择（下拉联动客户列表，支持搜索）、订单类型选择、设备选择（下拉联动设备列表，仅显示正常可用设备）、订单金额（保留 2 位小数，校验非负）、交货日期（校验不小于当前日期）、备注，表单验证：必填项不能为空、金额非负、日期合法，新增时校验客户+设备组合无重复订单（同客户同设备当日不可重复下单）。
4. 订单操作：支持单个订单审核/取消审核（仅管理员可操作）、单个/批量删除订单（仅未审核订单可删除），点击操作弹出确认弹窗，操作成功/失败给出明确提示，实时更新表格数据；支持订单详情查看（弹窗展示完整订单信息，包含关联的客户联系方式、设备规格）。
5. 核心要求：结合前端技术栈特性和业务需求，重点考察状态管理、路由控制、交互优化及异常处理，要求代码规范、性能优良，同时体现 AI 辅助开发的合理性和高效性，确保 20 分钟内完成核心功能落地。
6. 使用 Redux Toolkit 管理全局状态（登录信息、业务模块状态、订单分页/搜索/筛选参数、表单弹窗状态、客户/设备下拉数据缓存），实现状态持久化（localStorage），避免重复请求。
7. 路由守卫：使用 React Router v6 实现路由拦截，未登录用户无法访问除登录页外的所有页面，不同角色（管理员/普通操作员）可见菜单不同（普通操作员无审核、删除权限），token 过期自动跳转登录页。
8. 使用 AI 辅助实现表单联动逻辑（客户-设备联动）、订单筛选/分页逻辑、侧边栏业务模块切换动画，以及订单审核状态切换逻辑，标注 AI 辅助的代码片段及自身优化思路（重点优化数据联动性能、避免重复渲染）。
9. 异常处理：订单数据请求失败、表单提交失败、token 过期、权限不足，给出友好提示（使用 Ant Design 的 Message 组件），同时实现请求重试机制（最多 2 次重试），订单删除/审核失败需显示具体业务原因（如“该订单已审核，无法删除”）。

拓展问题（简要回答，无需展开）：1. React18 中 useMemo 和 useCallback 的区别，结合本业务场景说明使用场景？2. Redux Toolkit 如何实现不同业务模块的状态隔离？3. 如何优化前端下拉联动（客户-设备）的加载性能？

二、后端模块（35 分）

需求：基于 Java+Spring Boot+Spring Security+MyBatis-Plus，实现“客户订单管理后端接口”，聚焦客户订单核心业务，与前端页面完成联调，贴合真实业务逻辑，要求接口规范、逻辑严谨、具备高安全性，使用 AI 辅助实现接口开发、异常处理及权限控制，需在 25 分钟内完成核心实现。

1. 接口设计：遵循 RESTful 规范，结合业务关联逻辑，实现以下核心接口（需返回统一响应格式：{code: 状态码, message: 提示信息, data: 响应数据, timestamp: 时间

截})：

2. 用户认证接口：登录（POST /api/auth/login，接收用户名、密码，验证通过后返回 JWT token，包含用户角色（管理员/普通操作员）信息）、退出登录（POST /api/auth/logout，清除 token 及 Redis 缓存）。
3. 核心业务接口（业务关联）：围绕客户订单管理核心场景，实现订单全流程接口，关联客户、设备相关数据，确保接口逻辑贴合业务规则，支持前端所有业务操作，同时保证接口安全性和可扩展性。
4. 订单管理接口：作为核心业务接口，覆盖订单的查询、新增、编辑、审核、删除及详情查看，适配前端分页、筛选、搜索等需求，实现订单状态流转的业务逻辑，确保数据一致性和接口响应效率。
5. 获取订单列表（GET /api/orders，支持分页、多条件筛选、模糊搜索，参数校验，关联查询客户名称、设备信息）。
6. 新增订单（POST /api/orders，接收订单信息，校验客户、设备合法性，校验客户+设备组合当日不重复，返回新增订单详情）。
7. 编辑订单（PUT /api/orders/:id，仅未审核订单可编辑，接收修改后的订单信息，校验合法性，返回修改后详情）。
8. 订单审核/取消审核（PUT /api/orders/:id/status，仅管理员可操作，修改订单状态，返回操作结果）。
9. 删除订单（DELETE /api/orders，支持单个 ID、批量 ID 删除，仅未审核订单可删除，返回删除结果）。
10. 获取订单详情（GET /api/orders/:id，关联查询客户联系方式、设备规格，返回完整订单信息）。
11. 关联数据接口（支撑前端联动）：为前端表单联动、下拉选择提供数据支撑，确保客户、设备数据的实时性和准确性，减少前端冗余请求，优化交互体验，同时与核心订单接口形成业务联动。
12. 客户列表（GET /api/customers，支持模糊搜索，返回客户 ID、客户名称、联系方式，供前端下拉选择）。
13. 设备列表（GET /api/equipment，支持筛选正常可用设备，返回设备编号、设备名称、规格，供前端下拉选择）。
14. 核心要求（重点考察业务逻辑+技术能力，难度提升，非通用）：
15. 权限控制：使用 Spring Security+JWT 实现 token 认证，基于角色的权限控制（RBAC），管理员拥有全部操作权限，普通操作员仅可查看、新增订单，无法审

核、删除订单；除登录接口外，所有接口需验证 token 有效性，无效/过期返回 401 状态码，权限不足返回 403 状态码。

16. 数据校验：使用 JSR380 (javax.validation) 对所有接口的请求参数进行严格校验，结合业务规则添加自定义校验（如订单交货日期不小于当前日期、客户+设备组合当日不重复、未审核订单才可编辑/删除），校验失败返回 400 状态码及具体业务提示。

17. 业务逻辑：实现订单状态流转逻辑（待审核→已审核→已发货→已完成），禁止非法状态切换（如已审核订单不可直接删除）；关联查询优化，避免多表联查性能问题，新增订单时需校验客户、设备是否存在且状态正常。

18. 异常处理：实现全局异常处理器，捕获数据库查询失败、接口参数错误、token 过期、权限不足、业务逻辑异常（如非法订单状态切换）等异常，返回对应状态码及友好业务提示，打印关键异常日志（便于排查）。

19. 使用 AI 辅助实现接口逻辑、Spring Security+JWT 认证、全局异常处理、自定义业务校验，标注 AI 生成的核心代码及自身修改、优化的部分，说明优化原因（重点优化多表联查性能、业务逻辑严谨性、权限控制精细化）。

拓展问题（简要回答，无需展开）：1. Spring Boot 中如何实现不同业务模块的接口隔离？2. MyBatis-Plus 如何优化 PostgreSQL 的多表联查性能（结合本业务场景）？3. 如何防止后端接口的业务逻辑漏洞（如非法订单状态切换）？

三、数据库模块（20 分）

需求：基于 PostgreSQL+Redis，结合客户订单核心业务，设计订单关联数据库结构，实现数据持久化及缓存优化，结合后端接口完成数据交互，使用 AI 辅助设计表结构、编写 SQL 语句及 Redis 缓存逻辑，与前后端同步实现（贴合业务设计，避免通用表结构）。

20. PostgreSQL 表设计（难度提升，贴合业务，非通用）：结合客户订单管理业务，设计关联表结构，贴合 PostgreSQL 数据库特性，合理设置字段类型、主键、外键及索引，兼顾数据完整性和查询性能，避免通用化表设计，考察数据库设计能力。

21. 设计 4 张核心关联表（贴合业务，避免通用用户表）：订单表 (orders)、客户表 (customers)、设备表 (equipment)、用户角色表 (sys_user_role)，实现订单与客户、订单与设备的关联，用户与角色的关联，要求字段合理、类型贴合 PostgreSQL 特性、设置主键、外键及必要的索引（优化查询效率）。

22. 订单表 (orders)：订单 ID (主键, UUID, PostgreSQL 专属类型)、客户 ID (外键, 关联客户表)、设备 ID (外键, 关联设备表)、订单类型 (varchar, 采购/销售)、订单金额 (numeric(10,2), PostgreSQL 高精度类型)、交货日期 (date)、订单状态 (varchar, 待审核/已审核/已发货/已完成)、备注 (text)、创建

人 ID（关联用户表）、创建时间（timestamp）、更新时间（timestamp），设置索引：客户 ID+设备 ID+创建日期（联合索引，优化重复校验）、订单状态（普通索引）。

23. 客户表（customers）：客户 ID（主键，UUID）、客户名称（varchar，唯一）、联系人（varchar）、联系方式（varchar，唯一）、客户状态（varchar，正常/禁用）、创建时间（timestamp），设置索引：客户名称（唯一索引）、联系方式（唯一索引）。

24. 设备表（equipment）：设备 ID（主键，UUID）、设备编号（varchar，唯一）、设备名称（varchar）、设备规格（varchar）、设备状态（varchar，正常/维修/禁用）、创建时间（timestamp），设置索引：设备编号（唯一索引）、设备状态（普通索引）。

25. 用户角色表（sys_user_role）：用户 ID（主键，UUID）、角色名称（varchar，管理员/普通操作员）、角色描述（text）、状态（varchar，正常/禁用），设置索引：角色名称（普通索引）。

26. 使用 AI 辅助编写 PostgreSQL 表创建 SQL 语句，优化字段设计（贴合 PostgreSQL 特性，如 UUID 主键、numeric 金额类型、text 备注类型），标注 AI 优化的点及 PostgreSQL 与 MySQL 的差异适配。

27. Redis 缓存设计（重点考察，贴合业务缓存需求）：结合订单管理业务场景，设计合理的缓存策略，覆盖订单数据、关联数据及业务校验数据，实现缓存与数据库的同步，提升接口响应速度，考察缓存设计及性能优化能力。

28. 缓存订单数据：订单列表分页/筛选结果（缓存 key：orders:page:xxx，过期时间 10 分钟）、单个订单详情（缓存 key：order:id:xxx，过期时间 1 小时），订单新增/编辑/删除/状态变更后，同步更新或删除对应缓存。

29. 缓存关联数据：客户列表（缓存 key：customers，过期时间 6 小时）、正常设备列表（缓存 key：equipment:normal，过期时间 6 小时），客户/设备信息修改后，同步更新缓存，减少数据库查询压力。

30. 缓存业务校验数据：客户+设备组合当日订单缓存（缓存 key：orders:check:customerId:equipmentId:date，过期时间 24 小时），用于新增订单时快速校验重复，提升接口响应速度。

31. 使用 AI 辅助编写 Redis 缓存的增、删、改、查逻辑，结合 Java 后端接口实现缓存同步，标注 AI 生成的核心代码及自身优化思路，重点适配 PostgreSQL 的 UUID 类型缓存逻辑。

32. 额外要求：编写 2 条复杂 PostgreSQL 查询语句（贴合业务，非通用）：1. 查询近 30 天已审核的销售订单，关联客户名称、设备名称，按订单金额倒序排序，分页显示前 10 条；2. 统计各设备关联的订单数量（仅正常设备），过滤已取消/删除的订

单，按订单数量倒序排序。使用 AI 辅助优化 SQL 语句，结合 PostgreSQL 索引特性提升查询效率。

拓展问题（简要回答，无需展开）：1. PostgreSQL 中 UUID 主键相比自增主键的优势，结合本业务场景说明？2. PostgreSQL 的 numeric 类型与 MySQL 的 decimal 类型的差异，如何适配？3. Redis 如何缓存 PostgreSQL 的关联查询数据，避免缓存不一致？

四、综合联调与问题排查（10 分）

需求：完成前后端接口联调，确保订单管理关联业务功能正常运行，模拟线上业务相关问题排查，使用 AI 辅助排查并解决问题。

1. 前后端联调：确保前端所有业务操作（订单增删改查、审核、客户-设备联动、筛选分页）能正常调用后端接口，数据同步一致，无跨域问题（后端配置 CORS），PostgreSQL 数据库数据正常读写，Redis 缓存同步生效。
2. 问题排查（模拟 2 个业务相关高频问题，非通用问题，需快速定位并解决）：结合订单管理业务及技术栈特性，模拟线上常见故障场景，考察问题排查思路、定位能力及解决方案落地，同时结合 AI 辅助排查，提升效率。
3. 模拟场景 1：前端新增订单时，选择客户和设备后提交，后端返回 400 状态码，提示“客户+设备组合当日已存在订单”，但前端未显示该订单，使用 AI 辅助分析可能的原因（如缓存未同步、查询条件遗漏），并给出排查步骤及解决方案。
4. 模拟场景 2：前端查询订单列表时，无法显示设备名称（仅显示设备 ID），其他数据正常，使用 AI 辅助分析可能的原因（如多表联查异常、PostgreSQL 字段类型适配问题），并给出排查步骤及解决方案。
5. 核心要求：使用 AI 辅助排查问题，标注 AI 给出的排查思路，自身结合业务逻辑和代码逻辑定位问题并解决，说明问题原因及解决思路，重点贴合订单管理业务关联场景。

五、综合拓展（10 分）

1. 结合本次 60 分钟编程实现（客户订单管理业务），说明你如何使用 AI 工具提升开发效率，如何在短时间内完成前后端联调及 PostgreSQL 数据库适配，以及在使用 AI 过程中遇到的业务适配类问题（如 PostgreSQL 类型适配、业务逻辑漏洞），如何快速解决这些问题？
2. 针对本次实现的客户订单管理系统，提出 2 点以上的优化建议（结合业务场景，从性能、安全性、可扩展性方面），简要说明具体实现思路，可使用 AI 辅助给出优化方案（如订单批量导入、设备故障预警关联订单）。

评分标准（总分 100 分）

模块	评分要点	分值
前端模块	业务功能完整度、UI 交互、代码规范、 Redux /路由使用、异常处理、 AI 辅助合理性、20 分钟内完成核心实现，贴合订单管理业务	25
后端模块	接口规范、业务逻辑严谨性、权限控制、数据校验、异常处理、 AI 辅助合理性、代码可运行、25 分钟内完成核心实现，贴合订单管理业务	35
数据库模块	PostgreSQL 表结构设计（贴合业务）、 SQL 语句（业务相关）、 Redis 缓存逻辑（业务适配）、 AI 辅助优化、缓存同步，适配 PostgreSQL 特性	20
综合联调与问题排查	前后端联调成功率、业务相关问题排查思路、 AI 辅助排查合理性、快速解决问题能力，贴合订单管理业务场景	10
综合拓展	AI 使用总结、短时间开发效率把控、优化建议合理性（贴合业务场景）	10

备注：1. 代码无法运行、核心业务功能未实现，该模块按 0 分计；2. 未标注 **AI** 辅助场景及修改逻辑，扣 5-8 分；3. 代码无注释、格式混乱，扣 3-5 分；4. 未在规定时间内完成核心实现，酌情扣 5-10 分；5. 拓展问题未回答或回答不贴合订单管理业务，酌情扣分；6. 未适配 **PostgreSQL** 特性，扣 3-5 分。